

1. Check Execution Plan

Most important tuning technique.

PostgreSQL & MySQL

EXPLAIN ANALYZE

Oracle

EXPLAIN PLAN

Execution plan reveals:

- Table Scan
- Index Scan
- Join Type
- Cost

Without execution plan, SQL tuning becomes guesswork.

*EXPLAIN ANALYZE select * from customers;*

'-> Table scan on customers (cost=1.85 rows=16) (actual time=0.068..0.079 rows=15 loops=1)\n'

What happened?

MySQL performed a **full table scan** on the customers table.

Estimated by optimizer:

- cost=1.85 → Estimated cost of this operation.
- rows=16 → MySQL expected about 16 rows in the table.

Actual execution:

- actual time=0.073..0.082
 - Started at 0.073 ms
 - Finished at 0.082 ms
- rows=15
 - Actually read 15 rows.
- loops=1
 - This operation ran once.

*EXPLAIN ANALYZE select * from customers where first_name='Eric';*

'-> Filter: (customers.First_Name = 'Eric') (cost=1.85 rows=2) (actual time=0.058..0.063 rows=1 loops=1)\n -> Table scan on customers (cost=1.85 rows=16) (actual time=0.043..0.052 rows=15 loops=1)\n'

Estimated by optimizer:

- rows=2
 - Optimizer predicted approximately 2 rows would match.

Actual execution:

- rows=1
 - Only 1 row matched.
- actual time=0.045..0.049
 - Filtering took about 0.004 ms.



Why did MySQL do a Table Scan?

Most likely because there is **no index on First_Name** Column.

What if an index exists?

*EXPLAIN ANALYZE select * from customers where customer_id=103;*

'-> Rows fetched before execution (cost=0.00..0.00 rows=1) (actual time=0.000..0.000 rows=1 loops=1)\n'

customer_id is likely the **PRIMARY KEY** of the table.

Since MySQL knows:

- Primary keys are unique.
- The query is looking for exactly one row.
- The row can be found directly through the primary key index.

MySQL performs a **constant lookup (const access)** and fetches the row immediately.

Why actual time = 0.000?

actual time=0.000..0.000

The execution was so fast that the measured time rounds to zero milliseconds.

Performance Ranking

Generally:

Primary Key Lookup (Best)



Unique Index Lookup



Non-Unique Index Lookup



Range Scan



Full Table Scan (Worst)

2. Select Required Columns Instead of SELECT *

✗ **Bad** SELECT * FROM employees;

✓ **Better** SELECT emp_id, emp_name FROM employees;

Why it improves performance

Less Data Read

If a table has:

Column	Size
emp_id	4 bytes
name	50 bytes
address	200 bytes
photo	1 MB

Using SELECT * retrieves everything, including large columns.

Reduced Network Traffic

Database sends fewer bytes to application.

3. Avoid SELECT DISTINCT

✗ `SELECT DISTINCT department_id FROM employees;`

Why it is expensive

Database must:

1. Sort rows OR
2. Build hash table

to remove duplicates.

DB has to process all rows before returning unique values.

Solution : We can avoid DISTINCT if the selected columns form a unique combination and do not generate duplicate rows.

"For this, we need to identify the combination of columns that returns unique rows without using DISTINCT."

4. Use Explicit JOINS Instead of WHERE Joins

Old Style

```
SELECT *  
FROM emp e, dept d  
WHERE e.dept_id = d.id;
```

Modern Style

```
SELECT *  
FROM emp e  
INNER JOIN dept d  
ON e.dept_id = d.id;
```

Better readability

- Join conditions are separated from filtering conditions.

Less chance of accidental Cartesian joins

- In old-style syntax, if you forget a join condition:

```
SELECT * FROM employee e, department d;
```

This creates a Cartesian product (employees × departments), which can be extremely expensive.

5. Use WHERE Instead of HAVING

✗

```
SELECT dept_id, COUNT(*)  
FROM employee  
GROUP BY dept_id  
HAVING dept_id = 10;
```



```
SELECT dept_id, COUNT(*)  
FROM employee  
WHERE dept_id = 10  
GROUP BY dept_id;
```

Why?

Execution order:

```
FROM  
WHERE  
GROUP BY  
HAVING  
SELECT  
ORDER BY
```

With HAVING:

- Grouping happens first
- Filtering later

With WHERE:

- Rows filtered first
- Less data to group

Huge savings on large tables.

6. Avoid Too Many Joins

Joining many tables in a single query increases the amount of work the database must perform, which can impact performance.

Joins are not bad. Relational databases are designed for joins. The problem occurs when:

- Too many large tables are joined.
- Join columns are not indexed.
- Unnecessary tables are included.
- The query returns huge intermediate result sets.

In those cases, CPU, memory, and disk I/O usage can increase significantly, leading to slower query execution.

Recommendation

Join only tables whose data is actually required.

7. Operator Performance Order

=
>
>=
<
<=
LIKE
<>

General Idea

Equality predicates are most selective.

WHERE emp_id = 100 => can directly seek index.

Slower

WHERE salary > 50000 => needs range scan.

Much Slower

WHERE name LIKE '%john%' => leading wildcard prevents index seek.

Often Worst

WHERE status <> 'ACTIVE'

Many rows qualify.

Mainly, Index usefulness decreases.

8. Use LIMIT / TOP

PostgreSQL

SELECT * FROM orders LIMIT 10;

Why?

Database can stop reading early.

Instead of scanning: 10 million rows

it may stop after: 10 rows

depending on query.

Useful for:

- Testing
- Pagination
- Sampling

9. Use Wildcards at End Only

Good

WHERE name LIKE 'John%'

Index can be used.

Bad

WHERE name LIKE '%John'

or

WHERE name LIKE '%John%'

DB must inspect many rows.

Reason: B-tree indexes are ordered from left to right.

10. Use EXISTS Instead of IN (for Large Subqueries)

Example

```
WHERE id IN
(
  SELECT emp_id
  FROM orders
)
```

Better

```
WHERE EXISTS
(
  SELECT 1
  FROM orders o
  WHERE o.emp_id=e.id
)
```

Why?

EXISTS stops after first match.

IN may need larger intermediate result processing.

Modern optimizers often rewrite them similarly, but EXISTS is frequently better for large correlated lookups.

11. Use UNION ALL Instead of UNION

UNION

```
SELECT id FROM A UNION SELECT id FROM B;
```

Removes duplicates.

Requires:

- Sorting
- Hashing

UNION ALL

```
SELECT id FROM A UNION ALL SELECT id FROM B;
```

Simply concatenates results.

Much faster.

Use when duplicates are acceptable.